COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***Artificial Intelligence***
Instructor	***Mam Faiza***
Assignment no.	***04***

Student Details

Registration No.	***Name***
FA24-BSE-048	***HAROON KHAN***

1. Clearly define the variables, domains, and constraints of the problem.

1. Problem Definition

This program solves a Map Coloring Problem using a Constraint Satisfaction Problem (CSP) approach.
The goal is to assign colors to regions such that no two adjacent regions have the same color.

2. Variables, Domains, and Constraints

Variables

The variables represent the regions of the map:

- *A, B, C, D*

Domains

Each region can take one of the following colors:

- *Red*
- *Green*
- *Blue*

So:

- *A → {Red, Green, Blue}*
- *B → {Red, Green, Blue}*
- *C → {Red, Green, Blue}*
- *D → {Red, Green, Blue}*

Constraints

Adjacent regions cannot have the same color:

- *A → B, C*
- *B → A, C, D*
- *C → A, B, D*
- *D → B, C*

This means:

- *A ≠ B, A ≠ C*
- *B ≠ A, B ≠ C, B ≠ D*
- *C ≠ A, C ≠ B, C ≠ D*
- *D ≠ B, D ≠ C*

Code:

```python
# CSP solver for Map Coloring Problem
# Variables
variables = ['A', 'B', 'C', 'D']
# Domain (possible colors)
domains = {
    'A': ['Red', 'Green', 'Blue'],
    'B': ['Red', 'Green', 'Blue'],
    'C': ['Red', 'Green', 'Blue'],
    'D': ['Red', 'Green', 'Blue']
}
# Constraints (neighbors)
constraints = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
# Counter for steps
steps = 0
# Function to check if assignment is valid
def is_valid(variable, value, assignment):  1 usage
    for neighbor in constraints[variable]:
        if neighbor in assignment and assignment[neighbor] == value:
            return False
    return True
# Backtracking CSP solver
def backtracking(assignment):  2 usages
    global steps
    steps += 1
    # If all variables assigned
    if len(assignment) == len(variables):
```

```
            return assignment
        # Select unassigned variable
        for var in variables:
            if var not in assignment:
                current_var = var
                break
        # Try domain values
        for value in domains[current_var]:
            if is_valid(current_var, value, assignment):
                assignment[current_var] = value
                result = backtracking(assignment)
                if result:
                    return result
                # Backtrack
                del assignment[current_var]
        return None
    # Solve the CSP
    solution = backtracking({})
    # Display results
    print("Final Solution:", solution)
    print("Total Steps:", steps)
```

Output:

```
"C:\Users\DELL\PycharmProjects\AI LAB\.venv\Scripts\python.exe" "C:\Users\DELL\PycharmProjects\AI LAB\assignment 04.py"
Final Solution: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}
Total Steps: 5

Process finished with exit code 0
```

Show the process of constraint checking or how conflicts are resolved.

. Constraint Checking / Conflict Resolution

Example:

Suppose:

- *A = Red*
- *B = Red*

Since the constraint A ≠ B is violated, this assignment is not allowed.

Resolution:

The algorithm performs backtracking and removes the invalid assignment. It then tries an alternative value for B, such as Green or Blue, until a valid assignment is found.

In this way, conflicts are handled through systematic backtracking.

4. Discuss any limitations of your approach (e.g., complexity, cases where no solution exists).

Limitations of This Approach

1. **High computational complexity**
   Backtracking may need to explore a large number of possible combinations.
2. **Poor scalability**
   As the number of variables increases, the search space grows exponentially.
3. **No guaranteed efficiency**
   In the worst case, the algorithm may examine nearly all possible assignments before finding a solution.
4. **Possibility of no solution**
   If the constraints are overly restrictive, the algorithm may conclude that no valid assignment exists.